

旅途 (train)

by EmptySoulist

【题意简述】

给出 n 个长度为 L 的区间 $[l_i, r_i]$, 令 $a_i \sim [l_i, r_i]$, 得到集合 U 。

令 T 为 U 的任一子集, 现有变量 $X = 0$, 每次操作选择 T 的一个子集 S 并令 X 异或上 S 内的异或和, 操作 N 次, 对 $c \in [0, 2^m)$ 求 $X = c$ 的方案数。

$n \leq 2 \times 10^5, m \leq 22, L \leq 32, N \leq 10^9$, 答案对 998244353 取模。

【算法 1】

不妨先考虑 $L = 1$ 的情况, 此时 U 固定, 令集合 T 的线性基为 A , 设 $\mathbf{A}$ 为 A 的线性张成, 那么每一步选中 $\mathbf{A}$ 中任一元素的方案数均为 $2^{|T|-|A|}$

从而最终方案数为生成结果 X 为 $\mathbf{A}$ 中任一元素的方案数均为 $2^{N|T|-|A|}$ 。

不妨固定一个 T , 考虑其对于 X 的贡献, 为其线性张成 $\mathbf{A}$ 中每个 x 增加 $2^{N|T|-|A|}$, 记作 F_T

从而答案显然只和 $\mathbf{A}$ 有关, 可以证明本质不同的线性张成是 $O(2^{m^2/2})$ 级的, 枚举本质不同的线性张成可以得到 10pts。

【算法 2】

我们考虑计算 $\text{FWT}(F_T)$, 可以证明以下结论:

- $\text{FWT}(F_T)$ 的取值有且仅有 $2^{N|T|}, 0$ 两种可能的取值。

考虑计算 $\text{FWT}(F_T)$ 与计算 $2^{N|T|-|A|}\text{FWT}(\prod_{a \in A}(1+z^a))$ 的过程是等价的, 注意到 $\text{FWT}(1+z^a)$ 的取值仅有 0 或者 2, 从而 $\text{FWT}(\prod_{a \in A}(1+z^a))$ 的取值有且仅有 $2^{|A|}, 0$ 两种可能的取值。

不难观察到, 想要 $\text{FWT}(F_T)[z^i]$ 为 $2^{|A|}$ 当且仅当对于每个 $a \in A$ 都有 $\text{FWT}(1+z^a)[z^i]$ 为 2, 这意味着对于某个 i , $(-1)^{|i \text{ and } a|}$ 对于每个 a 的取值均为 1。

定义 $c(i, a)$ 为 0 表示 $(-1)^{|i \text{ and } a|}$ 为 1 称为 i 与 a 正交, 否则为 1。

则 $c(i, a) \oplus c(i, b) = c(i, a \oplus b)$, 这表明当 i 与 A 中每个元素正交时, i 对于张成 $\mathbf{A}$ 正交, 从而推出如下结论:

- 设 i 满足 $\text{FWT}(F_T)[z^i]$ 取值为 $2^{N|T|}$, 则 i 与 $\mathbf{A}$ 正交。

又由于 i 与 $\mathbf{A}$ 正交等价于 i 与 $\mathbf{A}$ 的线性基上的每个元素正交, 等价于 i 与 T 中每个元素正交。

从而我们希望求解的是：

$$\sum_{T \subseteq U} FWT(F_T)$$

考虑对于某个 i 而言，当 T 中全体元素 $a_j \in T$ 与 i 正交时， $FWT(F_T)[z^i]$ 的取值为 $2^{|T|}$ ，从而设与 i 正交的 a_j 构成集合 U_i ，则仅当 $T \subseteq U_i$ 时上式的值为 $2^{|T|}$ ，从而求和的结果简单概括为 $(1 + 2^N)^{|U_i|}$

从而我们希望知道对于每个 i 而言， U 中与其正交的 a_j 的数量，也即：

$$\sum_{j=1}^n [c(i, a_j) = 0] \Rightarrow \sum_{j=1}^n [(-1)^{|i \text{ and } a_j|} = 1]$$

可以通过对 $\sum z^{a_j}$ 做 FWT 再解方程对每个 i 求解，期望得分 25pts，通过进一步的列式求解的形式可以做到 $O(m2^{m+L}) \sim O(L^2 m 2^m)$ 不等，期望得分 45 ~ 75pts。

【算法 3】

考虑 $L \neq 1$ 的情况下，类似刚刚的算法，可以发现实际上对于 i 考虑求解全体 U 集下 T 集 FWT 的值的和，不难发现当 a_j 与 i 正交时，其贡献 $1 + 2^N$ ，否则贡献 1。

从而我们等价于对于每个 x 求解（记 $c(i, j)$ 为 $i \cdot j$ ）：

$$\prod (r_i - l_i + 1 + \sum_{j=l_i}^{r_i} 2^N [j \cdot x = 0])$$

将 $[j \cdot x = 0]$ 拆为 $\frac{1+(-1)^{|j \cdot x|}}{2}$ ，则改为：

$$\prod (L(1 + 2^{N-1}) + 2^{N-1} \sum_{j=l_i}^{r_i} (-1)^{j \cdot x})$$

这等价于求解：

$$[z^x] FWT \left(\prod (L(1 + 2^{N-1}) + 2^{N-1} \sum_{j=l_i}^{r_i} z^j) \right)$$

可以注意到对于任意 x 而言，每个 i 下 $(L(1 + 2^{N-1}) + 2^{N-1} \sum_{j=l_i}^{r_i} (-1)^{j \cdot x})$ 的取值仅有 L 种，如果能够知道每种取值有多少个，那么直接累乘再做 IFFT 还原即可。

我们事实上只关心：

$$[z^u] \sum_{i=0}^n \left[FWT \left(\sum_{j=l_i}^{r_i} z^j \right) = k \right]$$

$$[z^u] \sum_{i=0}^n \left[FWT \left(z^{l_i} \sum_{j=l_i}^{r_i} z^{j \oplus l_i} \right) = k \right]$$

$$[z^u] \sum_{i=0}^n \left[FWT(z^{l_i}) \cdot FWT\left(z^{l_i} \sum_{j=l_i}^{r_i} z^{j \oplus l_i}\right) = k \right]$$

当 $r_i - l_i + 1 = L$ 时, 本质不同的 $(x \oplus x, x \oplus (x+1), x \oplus (x+2) \dots x \oplus (x+L-1))$ 这样的 L 元组只有 $O(mL)$ 种, 证明参考 CF1408I。

我们考虑预处理出来这 mL 种 L 元组 FWT 后的结果, 这样对于每种元组和 l_i 的 pair, u 的答案要么是 k 要么是 $-k$ (而且固定), 我们实际上只需要统计有多少个 k 有多少个 $-k$ 即可。

于是对于这 mL 个元组, 将对应的 l_i 拿出来做一次 FWT 即可得出这一判定。于是可以算出每个二元组 $(u, k), (u, -k)$ 的对数, 统计答案转为对于每个 u 计算贡献即可。

复杂度 $O(m^2 L 2^m + N)$, 期望得分 $60 \sim 75$ pts

【算法 4】

考虑选定 b 使得 $B = 2^b \geq L$, 考虑将值域进行分块, 每 B 个为一块, 则此时可以观察到 $\sum_{j=l_i}^{l_i+L-1} z^j$ 最多跨越两个块。

事实上求和关于块的分布只取决于 $l_i \bmod B$ 的值, 对于某个 x , 设 $u = \frac{x}{B}, v = x \bmod B$, 而 $d = \frac{l_i}{B}, p = l_i \bmod B$, 则 FWT 系数告诉我们所求即:

$$[z^x] (-1)^{|u \cdot d|} \left(\sum_{j=p}^{B-1} (-1)^{|j \cdot v|} \right) + (-1)^{|u \cdot (d+1)|} \left(\sum_{j=0}^{p-1} z^{|j \cdot v|} \right)$$

我们希望对于每个 x 求出 n 个 l_i 得到的上式不同的结果的数量。首先注意到上式的取值事实上只关乎 $l_i \bmod B, x \bmod B$, 对于相同的 $l_i \bmod B$ 和 $x \bmod B$, 两个求和的结果可以预处理出来, 设为 a, b , 而不跨块的情况见下文讨论, 则上式的贡献仅有 $+a+b, +a-b, -a+b, -a-b$ 四类情况。

于是考虑枚举 $l_i \bmod B$, 考虑将所有 $l_i \bmod B$ 相同的 l_i 同时处理, 此时我们想知道其对于不同的 x , 对于相同的 $\frac{x}{B}$ 有贡献区别仅在 $x \bmod B$ 不同导致的 a, b 不同, 而对应的 $\pm$ 系数情况组数量是完全一致的, 而相同的 $l_i \bmod B$ 又导致这一系列的 l_i 对于每个 x 产生的 (a, b) pair 是完全一致的。

于是我们只需要对于每个 d 求出 $(-1)^{|u \cdot d|} + (-1)^{|u \cdot (d+1)|}$ 的四类对子数量, 不难发现上式等价于考虑 $[z^u] FWT(xz^d + yz^{d+1})$ 。注意到四类对子的总数为 $l_i \bmod B = t$ 的 l_i 的数量 N_t , 四类对子分别设为 S_0, S_1, S_2, S_3 。则对于 $\sum z^d, \sum z^{d+1}, \sum z^{d \oplus (d+1)}$ 做 FWT 分别给出了:

$$S_0 - S_1 + S_2 - S_3$$

$$S_0 + S_1 - S_2 - S_3$$

$$S_0 - S_1 - S_2 + S_3$$

于是可以解方程还原四类对子的数量。

这样对于相同的 $l_i \bmod B$ 可以快速计算对全体 $x \in [0, 2^m)$ 答案的贡献，每次需要做一个大小为 2^{m-b} 的 FWT，因此这一部分的复杂度是 $O(B * \frac{m2^m}{B}) = O(m2^m)$ 而计算贡献的时候每个 $l_i \bmod B$ 需要为 $[0, 2^m)$ 中的每个 x 计算一次快速幂，朴素做是 $O(B \log N * 2^m)$ ，注意到实际上只需要计算 $O(LN)$ 种可能的快速幂，因此可以预处理快速幂，每次计算贡献查表即可，复杂度 $O(B2^m)$ ，由 B, L 平级，复杂度为 $O(L2^m)$

当 $l_i \bmod B$ 不跨块时，即要求：

$$(-1)^{|u \cdot d|} \sum_{j=l_i \bmod B}^{j+L-1} (-1)^{|j \cdot v|}$$

枚举 $l_i \bmod B$ 之后，前者等价于求 $[z^u]FWT(z^d)$ 取 $-1, 1$ 的数量，直接加起来做 FWT 即可得到 $S_1 - S_{-1}$ ，而 $S_1 + S_{-1}$ 已知，从而可以轻易还原。而算式的后者仍然可以预处理，一共需要预处理出 L^2 个对子，总体复杂度 $O(L^3 + NL + (m + L)2^m)$

综上，可以在 $O(LN + (m + L)2^m)$ 的复杂度解决本题，期望得分 100pts。